

## **Trabajo Práctico N°2**

**Francisco Fornari - Agustín Pluda - Ramiro Wasilewki - Lautaro Christiansen -  
Ayrton Marinoni**



**Universidad Católica de Santiago del Estero  
Departamento Académico Rafaela**

**Ingeniería en Informática**

**Base de Datos II**

**Ing. Marcela Andrea Vera**

**Ing. Alejandro Aguirre**

**Ing. Georgina Festa**

**Rafaela**

**2024**

## Índice

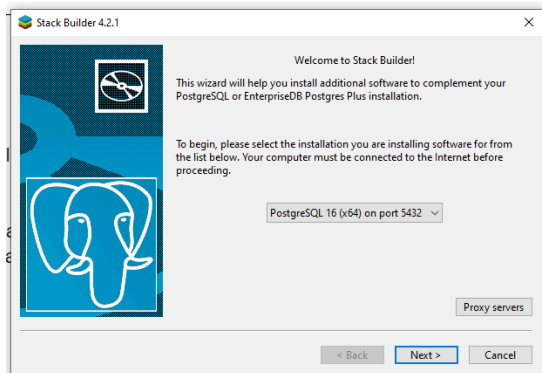
|                                                      |          |
|------------------------------------------------------|----------|
| <b>Ejercicio II.....</b>                             | <b>2</b> |
| Instalación:.....                                    | 2        |
| <b>Ejercicio III.....</b>                            | <b>3</b> |
| <b>Ejercicio V.....</b>                              | <b>4</b> |
| <b>Ejercicio VI.....</b>                             | <b>6</b> |
| <b>Ejercicio VII.....</b>                            | <b>6</b> |
| Bloqueos:.....                                       | 6        |
| Aislamiento de Transacciones:.....                   | 6        |
| ¿Esta BD permite trabajar en forma distribuida?..... | 6        |
| <b>Ejercicio VIII.....</b>                           | <b>7</b> |
| <b>Ejercicio IX.....</b>                             | <b>8</b> |
| Ventajas:.....                                       | 8        |
| <b>Ejercicio X.....</b>                              | <b>8</b> |
| Desventajas:.....                                    | 8        |
| <b>Ejercicio XI.....</b>                             | <b>8</b> |
| <b>Bibliografía.....</b>                             | <b>9</b> |

## Ejercicio II

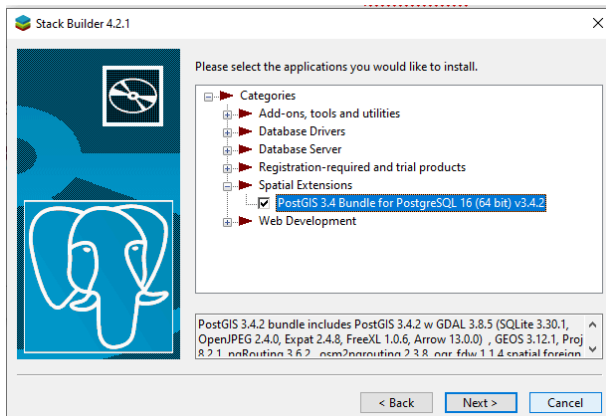
### Instalación:

Primero se instala PostgreSQL que es el software en el cual se puede instalar la extensión de Postgis.

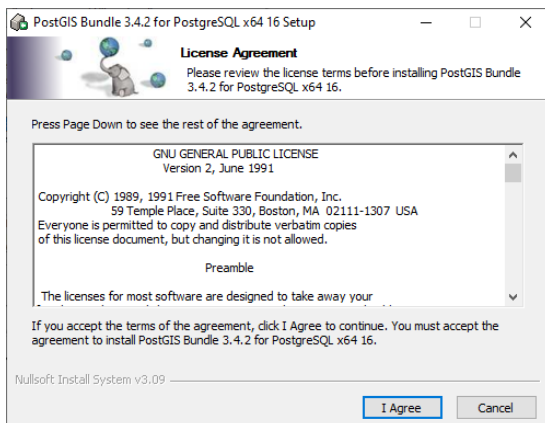
Luego de terminar la instalación, abrimos “Stack Builder” que es una de las aplicaciones que venía con el instalador, y con esa misma herramienta asignamos nuestro cliente de PostgreSQL.



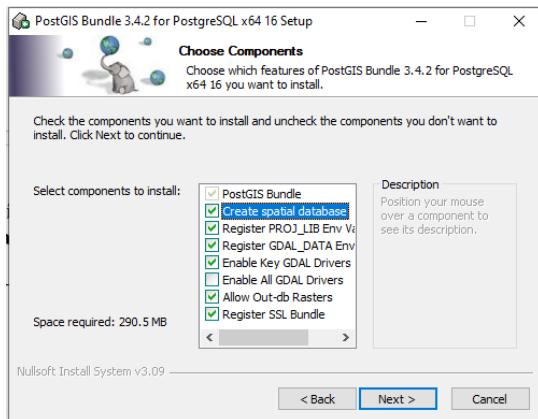
El paso siguiente es seleccionar en “Spatial Extensions” la aplicación PostGIS 3.4.



Luego, pide un sitio de instalación de la extensión, y se descarga. Al terminar la descarga pide los términos y condiciones de PostGIS, los cuales aceptamos.

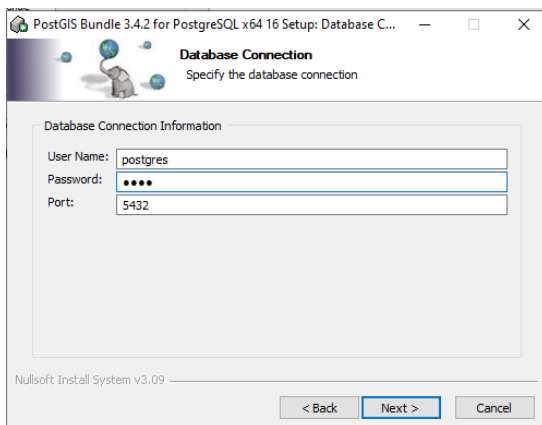


En las opciones de componentes, instalamos esto:



“Create spatial database” es para que cuando se instale PostGIS automáticamente se cree una base de datos espacial para tener de ejemplo, luego nos servirá para usarla de template)

Y por último, nos pide la información de conexión a la bdd.



## Ejercicio III

PostGIS maneja distintos tipos de datos espaciales en su sistema, algunos de los más básicos o importantes son estos:

1. Geometry y Geography:
  - Geometry: Este tipo de datos representa datos espaciales en un sistema de coordenadas proyectado. (por ejemplo, UTM, EPSG:4326).
  - Geography: Representa datos espaciales en coordenadas geográficas (latitud y longitud) sobre un elipsoide. Es útil para cálculos que deben tener en cuenta la curvatura de la Tierra, como cálculos de distancia más precisos.
2. Point:
  - Representa un punto en el espacio bidimensional o tridimensional. Por ejemplo, un punto de interés en un mapa.
3. Linestring:
  - Representa una secuencia de segmentos de línea que definen una forma lineal. Por ejemplo, un camino o una carretera.
4. Polygon:

- Representa un área cerrada definida por una secuencia de segmentos de línea que forman un perímetro. Por ejemplo, un país en un mapa.
- 5. MultiPoint, MultiLinestring, MultiPolygon:
  - Estos tipos de datos permiten almacenar colecciones de puntos, líneas o polígonos respectivamente. Por ejemplo, un conjunto de ciudades (MultiPoint), una red de carreteras (MultiLinestring), o un conjunto de regiones administrativas (MultiPolygon).
- 6. GeometryCollection:
  - Permite agrupar diferentes tipos de geometrías en una sola entidad. Puede contener cualquier combinación de Point, Linestring, Polygon, etc.

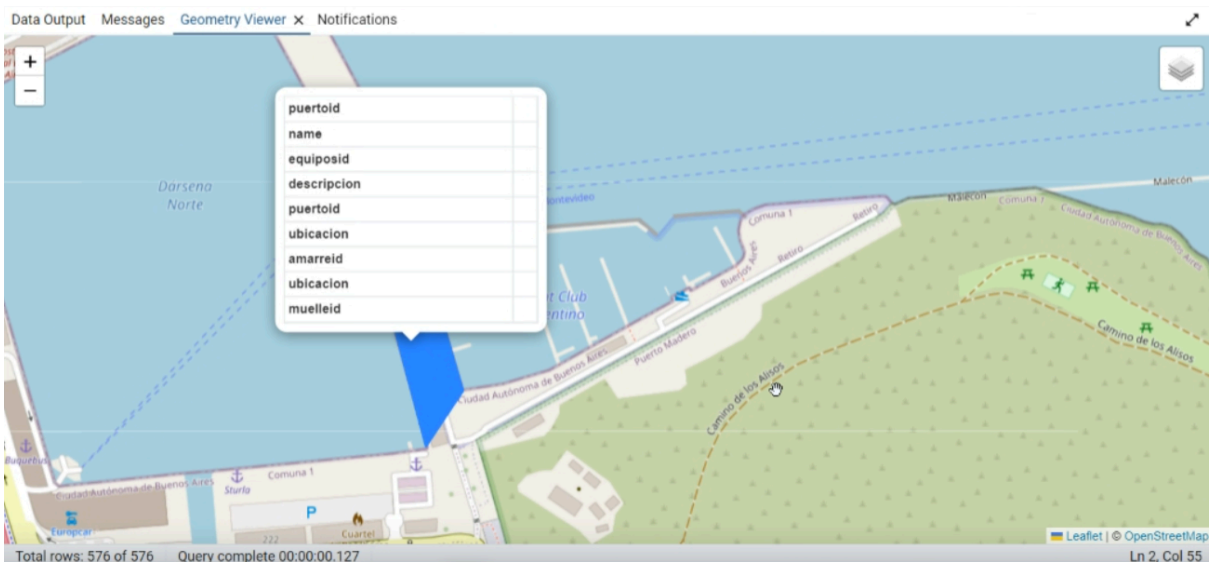
Además de los datos espaciales, PostGIS tiene una amplia gama de funciones espaciales avanzadas para realizar análisis espaciales complejos, como cálculos de distancia, intersecciones, uniones, bufferización, transformaciones de proyección, análisis topológicos, y más. Estas funciones están optimizadas para operar de manera eficiente incluso con grandes volúmenes de datos. Y para agregar, también cuenta con índices espaciales, que mejoran el rendimiento de consultas espaciales. Esto permite realizar búsquedas más rápidas y eficientes sobre conjuntos de datos geoespaciales grandes.

PostGIS también tiene una gran capacidad de extensión, y por esto mismo permite la integración con otras extensiones y funcionalidades de PostgreSQL, como funciones avanzadas de análisis de datos, soporte de transacciones ACID (Atomicidad, Consistencia, Aislamiento, Durabilidad), y capacidades de extensión para requisitos específicos del dominio.

## Ejercicio V

Para este ejercicio utilizamos una función de PostGIS llamada Transform, la cual al mandarle el campo Ubicación junto con el código 4326 que hace referencia a WGS84 que es un sistema geodésico de coordenadas geográficas, gráfica todas las tablas georeferenciadas. La sentencia que utilizamos fue

```
"SELECT ST_Transform(P.ubicacion, 4326), ST_Transform(E.ubicacion, 4326),  
ST_Transform(A.ubicacion, 4326)  
FROM puertos.puerto P, puertos.equipo E, puertos.amarres A"
```



## Ejercicio VI

- a) Dado un puerto, indique la cantidad de amarres disponibles, y la mínima distancia entre estos amarres libres:

Query Query History

```

1 SELECT
2   COUNT(*) AS amarres_disponibles,
3   MIN(ST_Distance(a1.ubicacion, a2.ubicacion)) AS min_distancia_entre_amarres
4 FROM
5   puertos.amarres a1
6   INNER JOIN puertos.puerto p ON ST_Contains(p.ubicacion, a1.ubicacion)
7   LEFT JOIN puertos.amarres a2 ON a1.amarreid <> a2.amarreid
8 WHERE
9   p.puertoid = 1
10  AND a1.amarreid NOT IN (Select a1.amarreid From puertos.amarresocupados a1)
11  AND a2.amarreid NOT IN (Select a2.amarreid From puertos.amarresocupados a2);

```

Data Output Messages Notifications

|   | amarres_disponibles | min_distancia_entre_amarres |
|---|---------------------|-----------------------------|
| 1 | 64                  | 0.00040323814303605505      |

Total rows: 1 of 1 Query complete 00:00:00.072

- b) Dado un barco que aún no ha llegado al puerto, calcular la distancia al puerto donde solicito servicio.

Query Query History

```

1 SELECT
2     b.name AS nombre_barco,
3     p.name AS nombre_puerto,
4     ST_Distance(b.ubicacion, p.ubicacion) AS distancia_en_metros
5 FROM
6     puertos.navio b,
7     puertos.puerto p
8 WHERE
9     b.navioid = 1
10 ORDER BY
11     distancia_en_metros;

```

Data Output Messages Notifications

|   | nombre_barco<br>character varying (80) | nombre_puerto<br>character varying (255) | distancia_en_metros<br>double precision |
|---|----------------------------------------|------------------------------------------|-----------------------------------------|
| 1 | Navio 1 - Amarrado                     | Buenos Aires                             | 6.774924447736365e-05                   |
| 2 | Navio 1 - Amarrado                     | San Nicolas                              | 2.1904473538937164                      |
| 3 | Navio 1 - Amarrado                     | Rosario                                  | 2.7726090043121796                      |
| 4 | Navio 1 - Amarrado                     | Ushuaia                                  | 22.52287060745943                       |

Total rows: 4 of 4 Query complete 00:00:00.197

- c) Muestre todos los barcos que han solicitado servicio a un puerto, y se encuentren dentro de un perímetro indicado.

Query Query History

```

1 SELECT n.name, n.ubicacion
2 FROM puertos.navio n
3 INNER JOIN puertos.puerto p ON ST_DWithin(p.ubicacion, n.ubicacion, 0.001)
4 WHERE
5     p.puertoid = 1
6     AND n.navioid IN (SELECT s.navioid
7                       FROM puertos.solicitud s
8                       WHERE s.puertoid = 1);

```

Data Output Messages Geometry Viewer x Notifications

|   | name<br>character varying (80) | ubicacion<br>geometry                              |
|---|--------------------------------|----------------------------------------------------|
| 1 | Navio 1 - Amarrado             | 0101000020E61000008E226B0DA52E4DC0826F9A3E3B4C41C0 |

Total rows: 1 of 1 Query complete 00:00:00.081

## Ejercicio VII

### Bloqueos:

En los posibles casos de bloques, se encuentran de ambos tipos, bloques compartidos y bloques exclusivos. El primero se usa cuando simplemente en la BD se quiere leer información. El segundo se utiliza a la hora de ingresar, modificar o borrar cualquier tupla del sistema, esto para evitar concurrencia con alguna otra transacción. Los casos anteriormente mencionados se tratan de protocolos, es decir, los pasos a seguir para evitar la concurrencia.

### Aislamiento de Transacciones:

Se utiliza un nivel de "REPEATABLE READ" cuando una transacción realiza una lectura de una o varias tuplas. Se usa este nivel porque al ser solo lectura, no hay posibilidad de que ocurra un caso de "Fantasmas", que este mismo ocurre solo cuando se ingresa una nueva fila, por lo tanto nos podemos permitir bajar el nivel de aislamiento para un mejor rendimiento. Y en todos los demás casos se usa un nivel de "SERIALIZABLE" para evitar la concurrencia totalmente, y no tener problemas entre transacciones.

### ¿Esta BD permite trabajar en forma distribuida?

Si, esta BD puede ser una Base de Datos Distribuida, PostgreSQL soporta configuraciones de BDD, y tiene ciertas características para trabajar de esta manera:

- Replicación: PostgreSQL soporta replicación de datos de manera síncrona y asíncrona. La replicación asíncrona es más común y permite a las bases de datos secundarias estar ligeramente desfasadas respecto a la principal, mientras que la replicación síncrona asegura que las transacciones están comprometidas en ambas bases de datos al mismo tiempo.
- Sharding y Particionamiento: Aunque PostgreSQL no soporta el sharding de manera nativa como algunas otras bases de datos distribuidas, se pueden usar herramientas como Citus, que es una extensión de PostgreSQL que permite escalar horizontalmente mediante el sharding de los datos a través de múltiples nodos.
- FDW (Foreign Data Wrappers): Permiten a PostgreSQL acceder a datos almacenados en otros sistemas de bases de datos, lo que puede ser útil para entornos distribuidos donde se necesita acceder a diversas fuentes de datos.

Dicho esto, PostgreSQL permite una configuración de BDD, y si queremos llevar nuestro caso a algo más realista, podríamos tener una base de datos en cada uno de los puertos en los que estamos enfocados y que por ejemplo, la BD del puerto de Buenos Aires sea centralizada y tenga las copias de Replicación de todos los demás puertos.

Y por último, PostgreSQL permite cambiar el nivel de aislamiento de transacciones con este tipo de comando:



Cambiar a READ COMMITTED:

SET TRANSACTION ISOLATION LEVEL READ COMMITTED;

Cambiar a READ UNCOMMITTED:

SET TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;

Cambiar a REPEATABLE READ:

SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;

Cambiar a SERIALIZABLE:

SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;

## Ejercicio VIII

Como caso de éxito tomamos en cuenta el mapa interactivo de la página de Ciudad de Buenos Aires. Esta misma utiliza PostgreSQL con PostGIS para mostrar estos mapas interactivos en sus sistemas. Tienen características de sistemas de información geográfica, las cuales son las siguientes:

- Integración de datos espaciales: Esta aplicación maneja una amplia variedad de datos geoespaciales, incluyendo coordenadas de puntos, líneas y polígonos que representan ubicaciones y formas geográficas para representar ciudades, calles, altura de la calles, etc.
- Análisis espacial: La página tiene varias características e información que surge del análisis de estos datos espaciales.
- Visualización cartográfica: Esta sección de aplicación nos permite visualizar el mapa de Buenos Aires.
- Interoperabilidad: La aplicación funciona en varios tipos de dispositivos, como el celular, computadora, etc.

Como conclusión, podemos tomar este ejemplo como un éxito en la implementación de PostGIS, el Mapa Interactivo de Buenos Aires es muy utilizado por sus ciudadanos y tiene muchas funcionalidades para poder guiarse por la ciudad.

## Ejercicio IX

### Ventajas:

- Integración de datos espaciales y no espaciales: Permiten integrar datos geográficos (como coordenadas, polígonos, líneas) con datos convencionales (textos, números, fechas), facilitando análisis complejos que combinan ambos tipos de datos.
- Puntos útiles: El mapa entre sus funciones, tiene varios tipos de mapas temáticos (como por ejemplo Hospitales, Farmacias, Postas Sanitarias, etc.) que nos ayudan a localizar puntos de necesidad con facilidad.
- Cálculo de distancia mas corta: En el sistema podemos seleccionar 2 direcciones, y un método de transporte para poder calcular y mostrar la distancia y demora más corta para llegar de un punto a otro.

- Rendimiento: Al ser simplemente una base de datos geográfica de Buenos Aires, va a tener un mejor rendimiento a comparación de una con escala mundial.
- Búsqueda de sitio: Dentro del mapa, nos permite la opción de búsqueda por escrito para buscar algún sitio, insertamos el nombre de algún lugar y nos proporciona resultados y ubicaciones de la búsqueda.

## Ejercicio X

### Desventajas:

- Consultas pesadas: Aunque las bases de datos geográficas están optimizadas para consultas espaciales, algunas operaciones pueden ser intensivas en términos de rendimiento, especialmente en bases de datos con grandes volúmenes de datos espaciales o en entornos con múltiples usuarios concurrentes.
- Complejidad en el diseño y mantenimiento: El diseño y mantenimiento de bases de datos geográficas puede ser más complejo que el de bases de datos tradicionales debido a la integración de datos espaciales y no espaciales, así como la gestión de funciones espaciales y de índices espaciales.
- Gran necesidad de almacenamiento: Al igual que los datos convencionales, los datos espaciales también ocupan almacenamientos y estos mismos se encuentran muchas veces en las tablas del sistema, por lo tanto las bases de datos geográficas necesitan más capacidad de almacenamiento que las bases de datos comunes.
- Actualización y mantenimiento de datos: Mantener la integridad y precisión de los datos geoespaciales puede resultar en procesos adicionales de actualización y mantenimiento, especialmente en entornos donde los datos cambian frecuentemente, como en aplicaciones de gestión urbana o ambiental.

## Ejercicio XI

PostGIS es recomendable para cualquier sistema donde la capacidad de manejar datos geoespaciales de manera eficiente y realizar análisis complejos basados en la ubicación sea crucial. Su integración con PostgreSQL también garantiza que se pueda beneficiar de todas las características y capacidades de una base de datos relacional robusta y de alto rendimiento.

Además de esto, PostGIS a comparación de las demás SIGs, es más simple, tiene menos funcionalidades, pero eso también significa que es un sistema menos complejo y que se ajusta más a la necesidades de alguna empresa mediana o chica.

Para resaltar, PostGIS es gratuito y de código abierto, con una solución robusta, escalable y compatible con estándares, lo que lo hace una opción atractiva para desarrolladores y analistas.

También ofrecen soporte para tipos de datos espaciales, como también tiene funciones espaciales avanzadas.

---

## Bibliografía

### Ejercicio III:

<https://postgis.net/>

<https://postgis.net/workshops/postgis-intro/>

### Ejercicio IV:

<https://epsg.io/map#srs=4326&x=-58.363763&y=-34.597050&z=18&layer=streets>

<https://www.argentina.gob.ar/puertos-vias-navegables-y-marina-mercante/Mapa-de-Puertos-Argentinos>

### Ejercicio VII:

<https://www.postgresql.org/docs/current/high-availability.html>

<https://www.citusdata.com/>

<https://www.postgresql.org/docs/current/sql-createforeigndatawrapper.html>

### Ejercicio VIII:

<https://mapa.buenosaires.gob.ar/comollego/?lat=-34.592732&lng=-58.499365&z=13&modo=pie>